

STUDY AND SELECTION OF SOFTWARE DEVELOPMENT LIFE CYCLE PROCESS MODEL

1. Selected Software Problem

Project Title:

An Explainable Graph-Enhanced Machine Learning Framework for Adaptive Anti-Money Laundering Detection

The proposed application is an intelligent Anti-Money Laundering (AML) detection and investigation platform. The system uses machine-learning algorithms to identify suspicious financial transactions and graph-based analysis to detect unusual relationships among bank accounts.

The application provides explainable risk predictions, dynamic customer-risk profiles, suspicious-transaction alerts, transaction-network visualization, and an investigation dashboard for AML analysts. Analysts can review generated alerts and classify them as confirmed suspicious, false positive, or requiring further investigation.

The proposed system consists of the following major components:

- Secure authentication and role-based authorization
- Transaction-data management
- Machine learning-based transaction-risk prediction
- Graph-based account-network analysis
- Explainable AI-based risk interpretation
- Dynamic customer-risk profiling
- Suspicious-activity alert generation
- AML investigation dashboard
- Human-in-the-loop analyst feedback
- Analytics and report generation

2. Study of SDLC Process Models

2.1 Waterfall Model

The Waterfall model is a linear and sequential software-development process. Development progresses through predefined phases, and each phase is generally completed before the next phase begins.

The major phases are:

Requirements Analysis → System Design → Implementation → Testing → Deployment → Maintenance

Advantages

- Simple and easy to understand
- Clearly defined development phases
- Well-defined project documentation
- Easy to manage when requirements are fixed
- Suitable for small projects with stable requirements

Limitations

- Difficult to incorporate changes after development begins
- Testing is performed relatively late
- Working software is available only after major development is completed
- Not suitable when requirements evolve frequently
- ML model performance cannot be fully predicted during the requirements phase

Suitability for the Proposed AML System

The Waterfall model is not highly suitable because the requirements of the AML system may change after evaluating the machine-learning model, graph-analysis methods, user feedback, and false-positive rates.

2.2 Spiral Model

The Spiral model is a risk-driven process model that combines iterative development with systematic risk analysis. The software is developed through multiple cycles called spirals.

Each spiral generally consists of:

Planning → Risk Analysis → Engineering → Customer Evaluation

Advantages

- Strong emphasis on risk identification and mitigation
- Supports changing requirements
- Suitable for large and complex systems
- Allows prototypes to be developed during different iterations
- Continuous stakeholder feedback is supported

Limitations

- Complex to manage
- Requires expertise in risk analysis
- Can increase development time and cost
- Not ideal for small academic projects
- Risk-assessment activities may create unnecessary overhead

Suitability for the Proposed AML System

The Spiral model is reasonably suitable because AML applications involve risks related to security, privacy, false predictions, and model reliability. However, it may be unnecessarily complex for a limited-duration academic project.

2.3 Agile Model

Agile is an iterative and incremental software-development approach. The application is divided into smaller functional components and developed through short development cycles called sprints.

A typical Agile cycle consists of:

Requirement Selection → Sprint Planning → Design → Development → Testing → Review → Feedback

Advantages

- Supports changing and evolving requirements
- Delivers working software incrementally
- Encourages continuous testing
- Supports regular stakeholder feedback
- Allows ML models and features to be improved iteratively
- Defects can be identified during each sprint
- High-priority features can be developed first

Limitations

- Requires continuous planning and collaboration
- Project scope may change frequently
- Accurate long-term estimation can be difficult
- Requires regular stakeholder participation
- Documentation may receive less attention if not managed properly

Suitability for the Proposed AML System

Agile is highly suitable because the AML system contains independent modules that can be developed and evaluated incrementally. The machine-learning model may require repeated experimentation, and system requirements may evolve according to prediction results and analyst feedback.

2.4 Prototype Model

The Prototype model involves developing an early working representation of the software to understand and refine user requirements.

The major phases are:

Initial Requirements → Prototype Design → Prototype Development → User Evaluation → Requirement Refinement → Final Development

Advantages

- Helps clarify unclear requirements
- Provides early visualization of the application
- Encourages user participation
- Improves usability through early feedback
- Reduces misunderstandings between users and developers

Limitations

- Users may assume that the prototype is the final product
- Frequent modifications may increase development time
- The initial prototype may have poor architecture
- Documentation may be neglected
- Not sufficient alone for managing ML experimentation

Suitability for the Proposed AML System

The Prototype model is useful for designing the AML dashboard, alert-investigation interface, transaction-network visualization, and explainability features. However, it does not provide a complete development approach for continuous ML-model evaluation and incremental system implementation.

2.5 Incremental Model

The Incremental model divides the application into multiple functional modules. Each increment adds new features to the existing system until the complete application is developed.

Example increments for the proposed application are:

Increment 1: Authentication and user management

Increment 2: Transaction management

Increment 3: Machine-learning risk prediction

Increment 4: Explainable AI and customer-risk profiling

Increment 5: Graph analysis and AML investigation dashboard

Advantages

- Working functionality is delivered early
- Features can be prioritized
- Testing is performed for every increment

- Development risks are reduced
- New functionality can be added gradually

Limitations

- Requires proper system architecture and planning
- Integration between increments may become complex
- Changes to the core architecture may affect previous increments
- Dependencies among modules must be carefully managed

Suitability for the Proposed AML System

The Incremental model is suitable because the AML platform consists of several independent but connected modules. However, Agile provides greater flexibility through continuous feedback and iterative improvement.

2.6 V-Model

The V-Model is an extension of the Waterfall model. Every development phase has a corresponding testing phase.

The relationship between development and testing is:

Requirements Analysis ↔ Acceptance Testing

System Design ↔ System Testing

Architecture Design ↔ Integration Testing

Module Design ↔ Unit Testing

Implementation is performed at the bottom of the V-shaped process.

Advantages

- Testing activities are planned early
- Strong focus on software quality
- Clearly defined development and testing phases
- Suitable for applications with stable requirements
- Provides high traceability between requirements and tests

Limitations

- Difficult to accommodate changing requirements
- Working software is produced relatively late
- Less flexible than iterative approaches
- Changes can increase development cost
- Not ideal for experimental ML systems

Suitability for the Proposed AML System

The V-Model provides strong testing and requirement traceability. However, the proposed system involves ML experimentation and evolving requirements, making a flexible and iterative model more appropriate.

3. Comparative Analysis of SDLC Models

Process Model	Requirement Flexibility	Risk Management	Early Working Software	Support for ML Experimentation	Suitability
Waterfall	Low	Low	Low	Low	Low
Spiral	High	Very High	Medium	High	High
Agile	Very High	High	Very High	Very High	Very High
Prototype	High	Medium	Very High	Medium	Medium–High
Incremental	High	Medium	High	High	High
V-Model	Low	Medium	Low	Low	Medium

4. Selected Process Model

Agile Software Development Model

The Agile process model is selected for developing the proposed Explainable Graph-Enhanced Machine Learning Framework for Adaptive Anti-Money Laundering Detection.

Justification

Agile is selected for the following reasons:

1. Evolving machine-learning requirements

The performance of the AML prediction model cannot be completely determined before experimentation. Different algorithms, features, pre-processing techniques, and risk thresholds may need to be evaluated. Agile supports continuous model improvement.

2. Incremental module development

The system can be divided into independent functional modules and developed across multiple sprints.

3. Continuous testing

Each module can be tested immediately after implementation. ML models can also be evaluated using precision, recall, F1-score, PR-AUC, ROC-AUC, false-positive rate, and false-negative rate.

4. Support for analyst feedback

AML analysts may provide feedback regarding alert quality, explanations, investigation workflow, and false-positive rates. Agile allows this feedback to be incorporated into future iterations.

5. Changing requirements

New transaction patterns, risk indicators, graph features, and explainability requirements may be identified during development. Agile supports requirement changes without restarting the entire project.

6. Early delivery of working modules

A functional version of the system can be demonstrated after every sprint instead of waiting until the complete application is developed.

7. Reduced development risk

Complex features such as graph analysis and explainable AI can be implemented after validating the core transaction-risk model

5. Proposed Agile Sprint Plan

Sprint	Major Activities	Expected Deliverable
Sprint 1	Requirement analysis, UI design, database design, authentication	Login system and initial database
Sprint 2	Customer and transaction management	Transaction monitoring module
Sprint 3	ML pre-processing, training, evaluation, and API integration	Suspicious-transaction prediction
Sprint 4	SHAP integration and dynamic customer-risk profiles	Explainable risk-analysis module
Sprint 5	Graph feature extraction and network visualization	Graph-based AML analysis
Sprint 6	Alert investigation, analyst feedback, reports, testing, and deployment	Complete AML platform

6. Conclusion

After studying the Waterfall, Spiral, Agile, Prototype, Incremental, and V-Model software-development process models, the Agile model was selected as the most suitable approach for the proposed AML application.

The system contains several independent modules and requires continuous experimentation with machine-learning algorithms, graph-analysis methods, risk thresholds, and explainability techniques. Agile supports incremental development, continuous testing, changing requirements, early delivery, and regular feedback. Therefore, Agile provides the flexibility required to develop and improve the proposed intelligent Anti-Money Laundering system efficiently.

Software Requirements Specification

for

An Explainable Graph-Enhanced Machine Learning Framework for Adaptive Anti-Money Laundering Detection

Version 1.0 approved

Prepared by Ankush Pratham (24BCE1937)

BCSE301L – Software Engineering

Faculty: Elakya R

14 July 2026

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Product Scope
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. External Interface Requirements

- 3.1 User Interfaces
- 3.2 Hardware Interfaces
- 3.3 Software Interfaces
- 3.4 Communications Interfaces

4. System Features

4.1 Authentication and Authorization

- 4.1.1 Description and Priority
- 4.1.2 Stimulus/Response Sequences
- 4.1.3 Functional Requirements

4.2 Transaction Ingestion and Monitoring

- 4.2.1 Description and Priority

4.2.2 Stimulus/Response Sequences

4.2.3 Functional Requirements

4.3 ML-Based Risk Detection

4.3.1 Description and Priority

4.3.2 Stimulus/Response Sequences

4.3.3 Functional Requirements

4.4 Graph-Based Analysis

4.4.1 Description and Priority

4.4.2 Stimulus/Response Sequences

4.4.3 Functional Requirements

4.5 Explainable Risk Analysis

4.5.1 Description and Priority

4.5.2 Stimulus/Response Sequences

4.5.3 Functional Requirements

4.6 Customer Risk Profiling

4.6.1 Description and Priority

4.6.2 Stimulus/Response Sequences

4.6.3 Functional Requirements

4.7 Alert Investigation and Feedback

4.7.1 Description and Priority

4.7.2 Stimulus/Response Sequences

4.7.3 Functional Requirements

4.8 Dashboard and Reporting

4.8.1 Description and Priority

4.8.2 Stimulus/Response Sequences

4.8.3 Functional Requirements

5. Other Nonfunctional Requirements

5.1 Performance Requirements

5.2 Safety Requirements

5.3 Security Requirements

5.4 Software Quality Attributes

5.5 Business Rules

6. Other Requirements

6.1 Database Requirements

6.2 Legal, Ethical, and Privacy Requirements

6.3 SDLC Process Model Selection

6.4 Study of Process Models

6.4.1 Waterfall Model

6.4.2 Spiral Model

6.4.3 Agile Model

6.4.4 Prototype Model

6.4.5 Incremental Model

6.4.6 V-Model

Appendix A: Glossary

Appendix B: Analysis Models

B.1 High-Level Architecture

B.2 Agile Increment Plan

Appendix C: To Be Determined List

Revision History

Name	Date	Reason for Changes	Version
Ankush Pratham	14 July 2026	Initial SRS	1.0

1. Introduction

1.1 Purpose

This SRS defines version 1.0 of the Explainable Graph-Enhanced Machine Learning Framework for Adaptive Anti-Money Laundering Detection. It specifies the complete web application, including transaction monitoring, ML risk prediction, graph analysis, explainable alerts, customer-risk profiling, investigation workflows, reporting, and analyst feedback.

1.2 Document Conventions

This document follows the supplied IEEE-style SRS template. Mandatory requirements use “shall.” Requirements use unique AML-FR identifiers. Priorities are High, Medium, or Low.

1.3 Intended Audience and Reading Suggestions

The intended readers are faculty evaluators, developers, testers, project managers, AML analysts, and maintainers. Sections 1–2 explain scope and context; Sections 3–4 define interfaces and functionality; Sections 5–6 specify quality, security, data, and operational requirements.

1.4 Product Scope

The product is an intelligent AML decision-support platform that combines transaction-level ML, account-network analysis, and explainable AI. It aims to improve alert prioritization, reduce avoidable false positives, support transparent investigations, maintain dynamic customer-risk profiles, and capture analyst feedback. It does not replace regulatory reporting or human compliance decisions.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- [Jensen and Iosifidis \(2022\), Fighting Money Laundering with Statistics and Machine Learning.](#)
- Lundberg and Lee (2017), A Unified Approach to Interpreting Model Predictions.

2. Overall Description

2.1 Product Perspective

This is a new, self-contained web application. A Next.js frontend communicates with a FastAPI backend. PostgreSQL stores users, customers, accounts, transactions, predictions, alerts, explanations, feedback, and audit records. An ML model produces transaction-risk probabilities, a graph engine computes network indicators, and SHAP provides explanations.

2.2 Product Functions

- Secure authentication and role-based access.
- Transaction import, validation, search, and monitoring.
- ML-based suspicious-activity prediction.
- Graph-based detection of cycles, rapid movement, and fan-in/fan-out patterns.
- Combined risk scoring and explainable alerts.
- Dynamic customer-risk profiles.
- Alert investigation, analyst feedback, dashboards, and reports.

2.3 User Classes and Characteristics

- Administrator: manages users, roles, configuration, model metadata, and audit access.
- AML Analyst: reviews alerts, explanations, profiles, and networks; records investigation outcomes.
- Authorized Data Operator: imports transaction data and reviews ingestion results.

2.4 Operating Environment

Modern browsers on Windows, macOS, or Linux; Next.js and TypeScript frontend; Python FastAPI backend; Supabase PostgreSQL; scikit-learn/XGBoost; SHAP; NetworkX; Vercel frontend and Render/Railway backend.

2.5 Design and Implementation Constraints

- Real labelled AML data may be restricted, requiring public or synthetic data.
- Sensitive communication shall use HTTPS.
- Frontend, API, ML, graph, and persistence modules shall remain separated.
- Predictions are decision-support outputs requiring human review.
- Academic timelines and cloud resources limit scale.

2.6 User Documentation

- Quick-start and navigation guide.
- Transaction-upload schema guide.
- AML analyst investigation guide.
- Administrator guide.
- In-application help for risk scores and explanations.

2.7 Assumptions and Dependencies

- Input data follows the documented schema.
- Model artifacts and preprocessing pipelines are compatible.
- Database, hosting services, and required libraries remain available.
- Users possess appropriate authorization and domain knowledge.

3. External Interface Requirements

3.1 User Interfaces

The responsive UI shall include login, dashboard, transaction monitor, customer-risk profile, alert queue, alert details, interactive network visualization, case-review form, feedback controls, reports, and administration pages. Tables shall support search, sorting, pagination, and filters. Risk shall use text labels in addition to visual indicators.

3.2 Hardware Interfaces

No specialized hardware is required. A standard computer, display, keyboard, pointing device, and internet connection are sufficient. The server requires adequate CPU, memory, and storage for inference, database operations, and graph processing.

3.3 Software Interfaces

- Next.js client to FastAPI REST API using JSON.
- FastAPI to Supabase PostgreSQL.

- FastAPI to trained ML model.
- ML output to SHAP explanation engine.
- Transaction data to NetworkX graph engine.

3.4 Communications Interfaces

All client-server communication shall use HTTPS and JSON. Authentication shall use secure tokens. APIs shall validate payloads, return standard HTTP status codes, and reject unauthorized requests.

4. System Features

4.1 Authentication and Authorization

4.1.1 Description and Priority

Provides secure access and role-based permissions.

Priority: High

4.1.2 Stimulus/Response Sequences

The authorized user initiates the feature; the system validates the request, performs the required analysis, stores the result, and displays a success response or an actionable error message.

4.1.3 Functional Requirements

AML-FR-01: The system shall authenticate registered users.

AML-FR-02: The system shall enforce role-based access.

AML-FR-03: Administrators shall manage users and roles.

4.2 Transaction Ingestion and Monitoring

4.2.1 Description and Priority

Imports, validates, stores, searches, and filters transaction records.

Priority: High

4.2.2 Stimulus/Response Sequences

The authorized user initiates the feature; the system validates the request, performs the required analysis, stores the result, and displays a success response or an actionable error message.

4.2.3 Functional Requirements

AML-FR-04: The system shall accept CSV or authorized API input.

AML-FR-05: The system shall validate identifiers, amounts, timestamps, and required fields.

AML-FR-06: The system shall provide search, sorting, filters, and ingestion errors.

4.3 ML-Based Risk Detection

4.3.1 Description and Priority

Calculates suspicious-activity probabilities and risk categories.

Priority: High

4.3.2 Stimulus/Response Sequences

The authorized user initiates the feature; the system validates the request, performs the required analysis, stores the result, and displays a success response or an actionable error message.

4.3.3 Functional Requirements

AML-FR-07: The system shall calculate a risk probability.

AML-FR-08: The system shall map probability to configurable risk categories.

AML-FR-09: The system shall store model version and prediction timestamp.

4.4 Graph-Based Analysis

4.4.1 Description and Priority

Analyzes account relationships and suspicious network patterns.

Priority: High

4.4.2 Stimulus/Response Sequences

The authorized user initiates the feature; the system validates the request, performs the required analysis, stores the result, and displays a success response or an actionable error message.

4.4.3 Functional Requirements

AML-FR-10: Accounts shall be nodes and transactions directed edges.

AML-FR-11: The system shall calculate degree, velocity, fan-in/fan-out, cycle, and neighborhood-risk indicators.

AML-FR-12: The system shall display an interactive filtered network.

4.5 Explainable Risk Analysis

4.5.1 Description and Priority

Provides interpretable factors and readable reasons for risk.

Priority: High

4.5.2 Stimulus/Response Sequences

The authorized user initiates the feature; the system validates the request, performs the required analysis, stores the result, and displays a success response or an actionable error message.

4.5.3 Functional Requirements

AML-FR-13: The system shall show factors increasing or decreasing risk.

AML-FR-14: The system shall generate readable explanations for high-risk alerts.

AML-FR-15: Explanations shall be stored for audit.

4.6 Customer Risk Profiling

4.6.1 Description and Priority

Maintains current and historical customer risk.

Priority: High

4.6.2 Stimulus/Response Sequences

The authorized user initiates the feature; the system validates the request, performs the required analysis, stores the result, and displays a success response or an actionable error message.

4.6.3 Functional Requirements

AML-FR-16: The system shall maintain a current customer-risk score.

AML-FR-17: The system shall display risk history and trends.

AML-FR-18: Risk shall be recalculated after significant activity.

4.7 Alert Investigation and Feedback

4.7.1 Description and Priority

Supports alert creation, review, case notes, outcomes, and feedback.

Priority: High

4.7.2 Stimulus/Response Sequences

The authorized user initiates the feature; the system validates the request, performs the required analysis, stores the result, and displays a success response or an actionable error message.

4.7.3 Functional Requirements

AML-FR-19: The system shall create alerts using configurable thresholds.

AML-FR-20: Analysts shall assign, comment on, and update alerts.

AML-FR-21: Analysts shall label outcomes as suspicious, false positive, or further investigation.

AML-FR-22: Feedback shall be stored for governed model improvement.

4.8 Dashboard and Reporting

4.8.1 Description and Priority

Summarizes AML operations and trends.

Priority: High

4.8.2 Stimulus/Response Sequences

The authorized user initiates the feature; the system validates the request, performs the required analysis, stores the result, and displays a success response or an actionable error message.

4.8.3 Functional Requirements

AML-FR-23: The dashboard shall show volumes, alert counts, risk distribution, and trends.

AML-FR-24: The system shall generate date-range reports.

AML-FR-25: Authorized users shall export permitted data.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Single predictions should return within 2 seconds under normal load.
- Common dashboard views should load within 3 seconds for the prototype dataset.
- Batch ingestion shall show progress and completion status.
- Evaluation shall report precision, recall, F1-score, PR-AUC, ROC-AUC, false-positive rate, and false-negative rate.

5.2 Safety Requirements

- The system shall not automatically accuse, block, or penalize a customer solely from an ML output.
- High-risk results shall require authorized human review.
- Destructive actions shall require confirmation.
- Failures shall not silently corrupt source data.

5.3 Security Requirements

- Passwords shall not be stored in plain text.
- HTTPS, secure tokens, input validation, least-privilege RBAC, and protected secrets shall be used.
- Sensitive data shall be restricted to authorized roles.
- Material actions and model decisions shall be audit logged.
- Common web vulnerabilities shall be mitigated.

5.4 Software Quality Attributes

- Availability: target 99% during scheduled testing.
- Reliability: recoverable errors without silent data loss.
- Usability: consistent navigation and understandable explanations.
- Maintainability: modular code, documented APIs, versioned models, and tests.
- Scalability: stateless API components should support horizontal scaling.
- Testability: requirements shall be traceable to test cases.

5.5 Business Rules

- Only administrators may manage roles and configuration.
- Only authorized analysts may view detailed evidence and submit outcomes.
- A prediction alone shall not constitute a final compliance decision.
- Thresholds and model versions shall be controlled and auditable.
- Feedback shall not automatically deploy a retrained model.

6. Other Requirements

6.1 Database Requirements

The database shall maintain users, customers, accounts, transactions, predictions, graph indicators, alerts, explanations, case notes, feedback, model metadata, and audit logs. Keys and constraints shall preserve integrity, and indexes shall support frequent account, date, status, and risk queries.

6.2 Legal, Ethical, and Privacy Requirements

The prototype shall use anonymized, public, or synthetic data wherever possible. Data shall be minimized and protected. Explanations and uncertainty shall be communicated to reduce overreliance on automated outputs.

6.3 SDLC Process Model Selection

Agile is selected because the system contains independently deliverable modules and requires repeated ML experimentation, threshold tuning, UI feedback, and continuous testing. Agile supports evolving requirements, early working increments, analyst feedback, and iterative improvement.

6.4 Study of Process Models

Model	Major Strength	Limitation for AML Project	Suitability
Waterfall	Clear sequential documentation	Poor support for evolving ML requirements	Low
Spiral	Strong risk analysis	High overhead for a lab project	High
Agile	Iterative delivery and feedback	Requires disciplined sprint planning	Very High
Prototype	Clarifies uncertain UI needs	Not a complete lifecycle strategy alone	Medium–High
Incremental	Early module delivery	Integration needs careful planning	High
V-Model	Strong verification and traceability	Rigid when model requirements change	Medium

Appendix A: Glossary

AML: Anti-Money Laundering

API: Application Programming Interface

KYC: Know Your Customer

ML: Machine Learning

PR-AUC: Area under the Precision–Recall curve

RBAC: Role-Based Access Control

SHAP: SHapley Additive exPlanations

XAI: Explainable Artificial Intelligence

Appendix B: Analysis Models

B.1 High-Level Architecture

Administrator / Data Operator / AML Analyst → Next.js Web Application → FastAPI Services → ML Model + Graph Engine + SHAP Engine → Supabase PostgreSQL → Alerts, Risk Profiles, Investigations, Feedback, and Reports.

B.2 Agile Increment Plan

- Sprint 1: requirements, UI, database, authentication.
- Sprint 2: customer and transaction management.
- Sprint 3: ML training, evaluation, and inference API.
- Sprint 4: SHAP explanations and customer-risk profiles.
- Sprint 5: graph analysis and visualization.
- Sprint 6: investigation, feedback, reports, testing, and deployment.

Appendix C: To Be Determined List

- TBD-1: Final public or synthetic AML dataset.
- TBD-2: Final risk thresholds after validation.
- TBD-3: Data-retention period.
- TBD-4: Final deployment provider and resource limits.
- TBD-5: Inclusion of a Graph Neural Network beyond the NetworkX baseline.